

JavaScript / Axios / JSON Server

10 Practice Project Ideas — Beginner to Advanced

A structured roadmap of hands-on CRUD projects covering
HTML, CSS, JavaScript, DOM, Axios, and JSON Server

📖 Includes Project Specs, Data Models, Features & Mandatory Folder-Structure Rules

 Overview of All Projects

#	Project	Difficulty	Main Concepts
1	👤🎓 Student Management System	☆☆	CRUD, DOM, Axios
2	📖 Library Management System	☆☆	CRUD, Search, Filters
3	💰 Expense Tracker	☆☆	CRUD, Calculations, DOM
4	🛒 Mini E-Commerce	☆☆☆	Products, Cart, CRUD
5	🎬 Movie Management System	☆☆☆	CRUD, Search, Categories
6	🎫 Ticket Booking System	☆☆☆	Booking, Validation, Status
7	👤🏢 Employee Management System	☆☆☆	CRUD, Filters, Dashboard
8	📋 Task Management / Kanban	☆☆☆	CRUD, Status, Drag & Drop
9	🏨 Hotel Booking System	☆☆☆☆	Multiple entities, Booking
10	💼 Job Portal	☆☆☆☆	Authentication, CRUD, Relations

Project Details

1. 👤🎓 Student Management System

Goal: Build an application to manage students.

Features

- Add student
- View all students
- Update student
- Delete student
- Search by name
- Filter by course
- View student details

Student Data Model

id	name
name	email
phone	age
course	city

API

GET	/students
POST	/students
PUT	/students/:id
DELETE	/students/:id

Best for: First Axios + JSON Server project.

2. 📖 Library Management System

Build a small library application where users can manage books.

Features

- Add book
- Edit book
- Delete book
- Search books
- Filter by category
- Issue a book
- Return a book
- Show available/issued books

Book Model

id	title
author	category
price	status (Available / Issued)

Endpoints

```
/books
/users
/issuedBooks
```

Good practice for: Multiple API endpoints.

3. 💰 Expense Tracker

A very practical project. Users can record their daily expenses.

Features

- Add expense
- Edit expense
- Delete expense
- Search expenses
- Filter by category
- Filter by date
- Calculate total expense
- Calculate category-wise expense

Expense Model

id	title
amount	category
date	description

Categories: Food, Travel, Shopping, Bills, Entertainment, Other

Sample Dashboard

```
Total Expense: ₹25,500
Food: ₹5,000
Travel: ₹7,000
Shopping: ₹8,000
Others: ₹5,500
```

Excellent project for practicing JavaScript calculations + DOM.

4. 🛒 Mini E-Commerce Application

Build your own small Amazon/Flipkart-style application.

Pages

- Home
- Products
- Product Details
- Cart
- Login
- Register

Product Model

id	name
price	category
image	description
stock	rating

Features

- Display products
- Search products
- Filter products
- Sort by price
- Product details
- Add to cart
- Remove from cart
- Change quantity
- Calculate total
- Place order

JSON Server Endpoints

```
/products
/users
/orders
```

Good intermediate project.

5. 🎬 Movie Management System

Create a Netflix/IMDb-style movie application.

Features

- Add movie
- Update movie
- Delete movie
- Search movie
- Filter by genre
- Sort by rating
- Movie details
- Mark as favourite

Movie Model

id	title
genre	language
releaseYear	rating
duration	description / poster

Example

```
Interstellar
Sci-Fi
English
2014
8.7
169 min
```

You can create a beautiful movie-card UI using HTML/CSS.

6. 🎫 Ticket Booking System

Build a simple movie/event ticket booking application.

Features

- Display available events
- View event details
- Select seats
- Book tickets
- Cancel booking
- View booking history

Event Model

id	name
location	date / time
price	availableSeats

Booking Model

id	userId / eventId
numberOfSeats	totalAmount
bookingDate	status

Example

```
Movie: Avengers
Seats: 3
Price: ₹250
Total: ₹750
```

Great for learning relationships between data.

7. 🏢 Employee Management System

Build an HR-style employee management application.

Employee Model

<code>id</code>	name / email / phone
<code>department</code>	designation
<code>salary</code>	joiningDate / status

Features

- Add employee
- Update employee
- Delete employee
- Search employee
- Filter by department
- Filter by designation
- Sort by salary
- Employee details
- Dashboard

Sample Dashboard

```
Total Employees: 120
IT: 50
HR: 20
Finance: 15
Sales: 35
```

You can also add: Active Employees / Inactive Employees

8. 📌 Task Management / Kanban Board

This one is very good for JavaScript DOM practice. Create a Trello-style application.

Columns

TODO → IN PROGRESS → COMPLETED

Task Model

<code>id</code>	title / description
<code>priority</code>	assignedTo
<code>status</code>	dueDate

Features

- Create task
- Edit task
- Delete task
- Change status
- Search task
- Filter by priority
- Filter by user
- Drag & Drop

Example Board

```
TODO
├─ Create Login Page
├─ Create Navbar

IN PROGRESS
├─ Create Dashboard

COMPLETED
├─ Setup JSON Server
├─ Create HTML
```

★ *Top recommendation.*

9. 🏨 Hotel Booking System

Build a mini hotel reservation application.

Hotel Model

id	name / location
rating	price / image
amenities	—

Room Model

id	hotelId
roomNumber	roomType
price	status

Booking Model

id	userId / hotelId / roomId
checkIn / checkOut	guests
totalAmount	status

Features

- Search hotels
- Filter by location
- Filter by price
- View hotel
- View rooms
- Book room
- Cancel booking
- Booking history

This teaches students how to work with multiple related resources.

10. 🏢 Job Portal

This is the most advanced one on the list. Think of a small version of Naukri/LinkedIn Jobs.

User Model

id	name / email
password	role

Job Model

id	title / company / location
salary	experience
skills	description / postedDate

Application Model

id	jobId / userId
resume	status / appliedDate

Features — Candidate

- Register
- Login
- Search jobs
- Filter jobs
- View job details
- Apply for job
- View applications

Features — Recruiter

- Login
- Create job
- Update job
- Delete job
- View applicants
- Change application status

Application Status Flow

Applied → Under Review → Shortlisted → Interview → Selected / Rejected

This project combines authentication + CRUD + Axios + JSON Server + DOM + multiple entities.

🔥 Recommended Learning Order

Tier	Projects
🟢 Beginner	1. Student Management → 2. Library Management → 3. Expense Tracker
🟡 Intermediate	4. Movie Management → 5. Employee Management → 6. E-Commerce
🔴 Advanced	7. Ticket Booking → 8. Kanban Board → 9. Hotel Booking → 10. Job Portal

★ If You Want ONE Project First

Choose **Student Management System** — it covers almost every core skill in sequence:

HTML → CSS → JavaScript → DOM → Form Validation → Axios → JSON Server → GET / POST / PUT / DELETE → Search → Filter → Pagination

Mandatory Folder Structure Rules

Every project listed above **must** follow this exact folder structure. This keeps code organized, separates concerns, and makes it easy to scale, debug, and review.

```
project-root/
├─ views/           # All HTML files go here
│  └─ index.html
│  └─ details.html
│  └─ ...
├─ css/            # All CSS / styling files go here
│  └─ style.css
│  └─ layout.css
│  └─ ...
├─ js/             # All JavaScript logic files go here
│  └─ service/      # Axios/API calls ONLY live here
│     └─ studentService.js
│     └─ apiConfig.js
│     └─ ...
│  └─ main.js       # DOM logic, event handlers
│  └─ utils.js
├─ exception/      # All error/exception handling files go here
│  └─ apiException.js
│  └─ validationException.js
│  └─ ...
├─ assets/         # Images, icons, fonts (optional)
└─ db.json         # JSON Server database
```

- ### 1. views/ — HTML Folder

 - Every .html file (pages, partials, templates) must be placed inside **views/**.
 - No HTML file should ever sit in the project root or inside **js/** or **css/**.
 - Page names should be lowercase and descriptive, e.g. `views/index.html`, `views/employee-details.html`.
- ### 2. css/ — Stylesheet Folder

 - All .css files must live inside **css/**.
 - No inline `<style>` blocks or inline `style="..."` attributes for anything beyond a quick one-off.
 - Split styles logically if the project grows (e.g. `css/style.css`, `css/dashboard.css`).
- ### 3. js/service/ — Service Folder (Axios / API Layer)

 - Every Axios call (GET, POST, PUT, DELETE) must be written inside a file in **js/service/** — never directly inside a DOM/event-handler file.
 - One service file per entity is preferred, e.g. `studentService.js`, `bookService.js`, `jobService.js`.
 - Service functions should only talk to the API and return data/promises — they must not touch the DOM.
 - Base URL / JSON Server config belongs in `js/service/apiConfig.js`.
- ### 4. js/ (root) — DOM & App Logic

 - DOM manipulation, event listeners, rendering, and form handling go in plain files directly inside **js/** (e.g. `main.js`), not inside `service/`.
 - These files call functions imported from `js/service/` — they do not write Axios calls themselves.
- ### 5. exception/ — Exception Handling Folder

 - All error handling, try/catch wrappers, custom error classes, and validation-error logic must be placed inside **exception/**.
 - Every service call should be wrapped in a try/catch that delegates error handling to a function from `exception/`, instead of handling errors inline everywhere.
 - Example files: `exception/apiException.js` (network/API errors), `exception/validationException.js` (form validation errors).

✔ Do	✗ Don't
Put every Axios call in <code>js/service/*.js</code>	Write <code>axios.get(...)</code> directly inside a button's onclick handler
Keep all HTML pages inside <code>views/</code>	Leave HTML files scattered in the project root
Wrap API/service calls in <code>exception/</code> handlers	Use bare try/catch blocks with <code>console.log</code> only, repeated everywhere

Keep CSS fully inside css/	Use inline style="" attributes throughout HTML
Name service files by entity: userService.js, jobService.js	Dump all API logic into one giant script.js

Summary rule of thumb: If it's markup → views/. If it's a style → css/. If it's an Axios/API call → js/service/. If it's DOM/UI logic → js/. If it's error handling → exception/. Nothing goes loose in the project root except db.json.